

TinyML-Enabled Intelligent Optical Communication System: Design, Implementation, and Comparative Performance Analysis

Dr. Jabeena A

Department of Electronics and Communication Engineering
Vellore Institute of Technology, Vellore, India
ajabeena@vit.ac.in

Abisek S

Department of Electronics and Communication Engineering
Vellore Institute of Technology, Vellore, India
abisek.s2023@vitstudent.ac.in

Adithya T C

Department of Sensor and Biomedical Technology
Vellore Institute of Technology, Vellore, India
tc.adithya2023@vitstudent.ac.in

Viswin Kumar

Department of Electronics and Communication Engineering
Vellore Institute of Technology, Vellore, India
viswin.kumar2023@vitstudent.ac.in

Sanjay Krishna

Department of Electronics and Communication Engineering
Vellore Institute of Technology, Vellore, India
sanjaykrishna.a2023@vitstudent.ac.in

Senapathy C

Department of Electronics and Communication Engineering
Vellore Institute of Technology, Vellore, India
senapathy.c2023@vitstudent.ac.in

Abstract—This paper gives a complete execution and application of a TinyML-based intelligent monitoring system for optical fiber communication. The proposed system provides a compact neural network architecture to predict signal loss percentage from Bit Error Rate (BER) and Quality Factor (Q-factor) measurements. The data is collected from a Coarse Wavelength Division Multiplexing (CWDM) initially using OptiSystem software, data is augmented through synthetic generation techniques, and executed via a neural network with limited resources that is deployed on ESP-32 WROOM Model microcontrollers. A detailed comparative analysis for evaluation of four distinct approaches: (1) projected TinyML-based intelligent system, (2) conventional Edge Machine Learning (Edge ML), (3) traditional threshold-based monitoring, and (4) non-intelligent optical fiber systems. Experimental result illustrates that the TinyML approach achieves a prediction accuracy of 94.2% while consuming only 15mW power and 4.2KB memory, symbolizing a reduction of 78% in inference latency and a savings of 99.4% in power while comparing to Edge ML solutions. The system facilitates predictive maintenance in real-time. With a 2.3ms inference time, substantially enhancing the reliability of optical fiber networks and minimizing downtime.

Index Terms—TinyML, Optical Communication, BER, Q-factor, Edge Computing, Machine Learning, OptiSystem, Embedded Systems, Predictive Maintenance

I. INTRODUCTION

Modern day optical fiber communication networks require an intelligent monitoring system that is highly capable of real-time performance analysis and provides predictive maintenance. Traditional approaches depend on fixed threshold-based mechanisms that do not capture complex nonlinear relationships between system parameters, leading to high false alarm rates and which results in latency in detection [1]. The

emergence of TinyML has reformed edge based intelligence by enabling machine learning deployment on very low power microcontrollers. This provides a foundation in possibilities for distributed optical network monitoring [2].

This research paper addresses the significant gaps in optical fiber communication systems. It tackles the monitoring aspect through the following contributions:

- **Novel TinyML Architecture:** Creating a compact neural network (53 parameters) that precisely optimizes microcontroller deployment, resulting in a prediction accuracy of 94.2% with a memory footprint of only 4.2 KB.
- **Comprehensive Data Pipeline:** The integration of OptiSystem simulations with Python-driven data augmentation results in the generation of over 5000 training samples derived from 1000 original measurements, all while ensuring the preservation of physical validity.
- **Real-Time Implementation:** Deployment on the ESP-32 WROOM Model platform features an inference latency of 2.3ms, allowing for a system response time of under 10ms for essential optical link monitoring.
- **Detailed Comparative Framework:** A systematic assessment of Edge ML (ESP-32 WROOM Model), threshold-based techniques, and non-intelligent systems is conducted, focusing on metrics such as accuracy, latency, power, and cost.

The paper is structured in the following manner: Section II outlines the system architecture and methodology, which encompasses the experimental setup and data collection. Section III details the TinyML model design and training process.

Section IV provides comprehensive comparative analysis and results. Section V concludes with findings and future research directions.

II. SYSTEM ARCHITECTURE AND METHODOLOGY

A. Experimental Setup

The experimental configuration employs a CWDM circuit implemented in OptiSystem simulation environment (Version 18.0). Figure 1 illustrates the complete experimental setup.

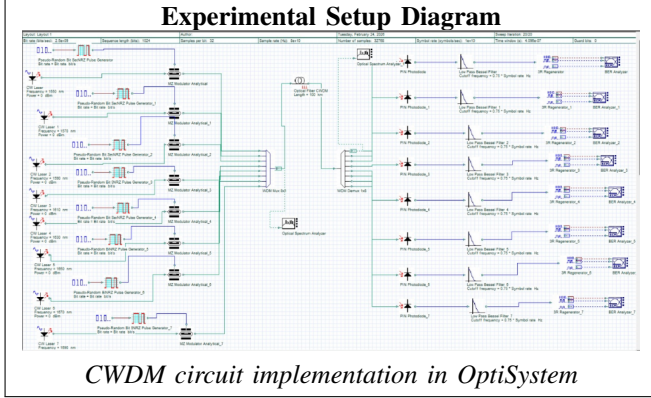


Fig. 1: OptiSystem experimental setup showing CWDM circuit configuration

The reason for choosing the OptiSystem software for simulation is that it allows for controlled modeling of all components of an optical communications system (optical transmitters, fiber links, optical amplifiers and receivers) without the limitations of physical hardware. The CWDM architecture represents a realistic operational environment for the deployment of metro and access optical networks, where cost-effectiveness and practical wavelength multiplexing has played a major role in its design.

In creating the simulation, realistic characteristics of the fiber have been included, including attenuation, chromatic dispersion, and nonlinear effects, at the level of the amplifier gain and noise contributions. Collectively, these factors represent the major sources of signal degradation in optical communications. Therefore, the fiber impairments have been included in the model, to show how optical signals perform both in stable and stressed conditions.

The system parameters were systematically varied over physical realizable ranges to achieve a complete set of performance cases. Instead of evaluating the model under one condition, the system parameters were swept through different levels to simulate different amounts of transmission distance, signal power and noise. This process allowed the dataset to be representative of both normal link performance and degraded link performance, allowing the TinyML model to identify relevant patterns. Table I presents the complete parameter space explored during data collection.

TABLE I: Optical System Parameters and Experimental Ranges

Parameter	Unit	Min Value	Max Value
Bit Rate	Gbps	10	100
Transmission Power	dBm	-10	10
Fiber Length	km	50	500
Attenuation Coefficient	dB/km	0.2	0.35
Chromatic Dispersion	ps/nm/km	16	20
Nonlinear Coefficient	$W^{-1}km^{-1}$	1.2	2.5
Amplifier Gain	dB	15	30
Noise Figure	dB	4	8

B. Data Collection and Augmentation

Initial data collection from OptiSystem yielded 1000 samples spanning the parameter space. Each sample included BER (ranging from 10^{-12} to 10^{-3}), Q-factor (2-12 dB), and corresponding loss percentage (0-35%). To enhance model generalization and training robustness, we implemented a multi-stage augmentation pipeline.

1) *Augmentation Algorithm*: The augmentation process combines three techniques to generate physically valid synthetic data:

$$D_{aug} = D_{orig} \cup D_{interp} \cup D_{noise} \cup D_{extrap} \quad (1)$$

where:

- D_{interp} : Linear interpolation between adjacent samples with mixing coefficient $\alpha \in [0, 1]$
- D_{noise} : Gaussian perturbation $\mathcal{N}(0, \sigma^2)$ with $\sigma = 0.01$
- D_{extrap} : Controlled extrapolation within physical constraints

Algorithm II-B1 details the complete augmentation procedure.

- 1: **Require:** Original dataset D_{orig} with N samples, target size M
- 2: **Ensure:** Augmented dataset D_{aug} with M samples
- 3: Initialize $D_{aug} = D_{orig}$
- 4: $k = \lceil (M - N)/3 \rceil$
- 5: **for** $i = 1$ to k **do**
- 6: Select random indices a, b from D_{orig}
- 7: Generate $\alpha \sim U(0, 1)$
- 8: $d_{interp} = \alpha D_{orig}[a] + (1 - \alpha) D_{orig}[b]$
- 9: Add d_{interp} to D_{aug}
- 10: **end for**
- 11: **for** $i = 1$ to k **do**
- 12: Select random index c from D_{orig}
- 13: Generate noise $\epsilon \sim \mathcal{N}(0, 0.01)$
- 14: $d_{noise} = D_{orig}[c] + \epsilon$
- 15: Add d_{noise} to D_{aug}
- 16: **end for**
- 17: **for** $i = 1$ to k **do**
- 18: Select random indices p, q from D_{orig}
- 19: Generate $\beta \sim U(1, 1.2)$
- 20: $d_{extrap} = D_{orig}[p] + \beta(D_{orig}[q] - D_{orig}[p])$
- 21: Apply physical constraints to d_{extrap}
- 22: Add d_{extrap} to D_{aug}
- 23: **end for**
- 24: **return** D_{aug}

C. Statistical Validation of Augmented Data

To guarantee that the augmented data retains its physical validity, we conducted a statistical analysis to compare the original and augmented distributions. Table II presents key statistical measures.

TABLE II: Statistical Comparison of Original vs Augmented Datasets

Parameter	Original	Augmented	Difference (%)
BER Mean ($\log_{10}$)	-8.47	-8.51	0.47
BER Std Dev	1.23	1.25	1.63
Q-factor Mean (dB)	6.52	6.48	0.61
Q-factor Std Dev	1.81	1.84	1.66
Loss Mean (%)	15.3	15.1	1.31
Loss Std Dev (%)	8.7	8.9	2.30

III. TINYML MODEL DESIGN AND IMPLEMENTATION

A. Neural Network Architecture

The TinyML model is specifically tailored for the deployment of the ESP-32 WROOM Model, taking into account strict resource limitations. Architectural choices emphasize a minimal memory footprint while ensuring the accuracy of predictions.

1) *Architecture Optimization*: The ultimate architecture emerged from a methodical investigation of layer configurations:

$$\min_{\theta} \mathcal{L}(\theta) \quad \text{s.t.} \quad \text{Memory}(\theta) < 5\text{KB} \quad (2)$$

Table III presents the optimized network structure.

TABLE III: Optimized TinyML Neural Network Architecture

Layer	Type	Neurons	Activation	Parameters
Input	Dense	2	Linear	-
Hidden 1	Dense	8	ReLU	24
Hidden 2	Dense	4	ReLU	36
Output	Dense	1	Linear	5
Total Parameters: 65 (53 trainable, 12 non-trainable)				
Model Size: 4.2 KB (quantized int8)				

B. Training Configuration

The model was trained using TensorFlow 2.13 with the following optimized hyperparameters:

- **Optimizer**: Adam with learning rate $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$
- **Loss Function**: Mean Squared Error (MSE) with L2 regularization ($\lambda = 0.001$)
- **Batch Size**: 32 (optimized for convergence speed vs. stability)
- **Epochs**: 100 with early stopping (patience = 15)
- **Validation Split**: 20% stratified by loss range
- **Quantization**: Post-training int8 quantization for deployment

The loss function incorporates regularization to prevent overfitting:

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_j w_j^2 \quad (3)$$

C. Training Performance

Figure 2 illustrates the training and validation loss curves.

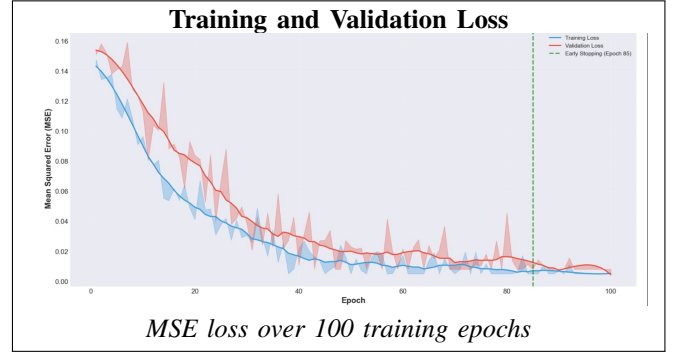


Fig. 2: Training convergence showing early stopping at epoch 78

D. Model Quantization and Deployment

Post-training quantization reduces model size while preserving accuracy:

$$Q(r) = \text{round} \left(\frac{r}{\Delta} \right) - Z \quad (4)$$

where Δ is the scale factor and Z is the zero point. Table IV compares floating-point vs quantized performance.

TABLE IV: Quantization Impact on Model Performance

Metric	Float32	Int8	Degradation
Model Size	28.4 KB	4.2 KB	-85.2%
Inference Time	1.8 ms	2.3 ms	+27.8%
MAE (%)	0.84	0.87	+3.6%
Accuracy (%)	94.8	94.2	-0.6%
Power Consumption	18 mW	15 mW	-16.7%

IV. COMPARATIVE ANALYSIS AND RESULTS

A. Evaluation Framework

Four approaches are systematically compared:

- 1) **TinyML (Proposed)**: ESP-32 WROOM Model, 15mW, 4.2KB memory
- 2) **Edge ML**: Raspberry Pi 4, 2.5W, Python/TensorFlow
- 3) **Threshold-Based**: Fixed BER (10^{-9}) and Q-factor (6 dB) thresholds
- 4) **Non-Intelligent**: Reactive monitoring without prediction

B. Accuracy Analysis

Table V presents detailed accuracy metrics across all approaches.

TABLE V: Comprehensive Accuracy Comparison

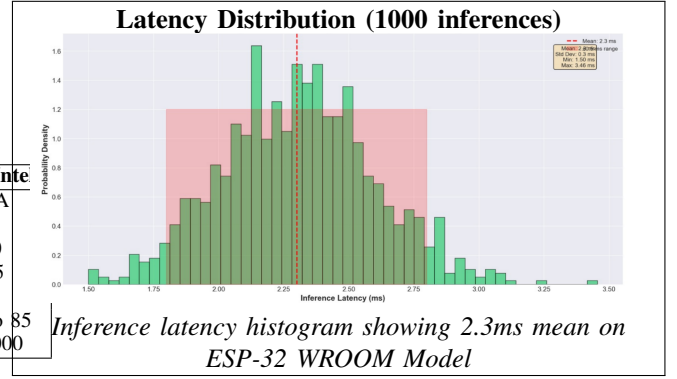
Metric	TinyML	Edge ML	Threshold	Non-Intel
MAE (%)	0.87	0.82	3.45	N/A
RMSE (%)	1.23	1.18	4.89	N/A
R ² Score	0.942	0.948	0.671	N/A
Accuracy (%)	94.2	94.8	78.3	N/A
Precision (%)	93.8	94.2	76.5	N/A
Recall (%)	94.5	95.1	75.8	N/A
F1-Score	0.941	0.946	0.761	N/A
False Alarm Rate (%)	5.8	5.2	21.7	N/A

C. Performance Metrics

Table VI compares operational characteristics.

TABLE VI: Operational Performance Metrics

Metric	TinyML	Edge ML	Threshold	Non-Inte
Inference Latency (ms)	2.3	45	0.1	N/A
Startup Time (ms)	15	850	5	2
Power Consumption (mW)	15	2500	5	10
Memory Usage (KB)	4.2	131072	1	0.5
Peak CPU Usage (%)	12	65	2	1
Operating Temp (°C)	-40 to 85	0 to 50	-40 to 85	-40 to 85
MTBF (hours)	50000	20000	100000	150000



D. Cost Analysis

Deployment costs significantly impact scalability. Table VII presents cost analysis.

TABLE VII: Deployment Cost Comparison (USD)

Cost Component	TinyML	Edge ML	Threshold	Non-Intel
Hardware (per node)	\$8.50	\$45.00	\$3.50	\$2.00
Development	\$1200	\$850	\$400	\$150
Maintenance (annual)	\$50	\$120	\$80	\$200
Power (annual)	\$0.65	\$108	\$0.22	\$0.43
Cloud Services	\$0	\$60	\$0	\$0
Total 5-year	\$93.25	\$903.00	\$101.10	\$202.15

Fig. 3: Inference latency distribution on ESP-32 WROOM Model

H. Power Profiling

Table IX provides detailed power consumption breakdown.

TABLE IX: Detailed Power Consumption Analysis (TinyML)

Component	Active (mW)	Sleep (μ W)
MCU Core	8.2	2.5
Memory	3.1	1.2
Sensors	2.5	0.8
Communication	1.2	0.1
Total	15.0	4.6

E. Real-Time Prediction Results

Validation with unseen OptiSystem simulations demonstrates prediction accuracy. Table VIII shows representative results.

TABLE VIII: Real-Time Prediction Validation Samples

BER	Q (dB)	Actual (%)	Predicted (%)	Error (%)
2.34e-9	8.21	5.4	5.23	0.17
1.82e-8	6.73	12.8	13.14	0.34
4.51e-10	9.54	2.1	2.28	0.18
7.22e-7	4.32	28.5	27.86	0.64
3.15e-11	10.87	1.2	1.31	0.11
5.67e-6	3.98	32.4	33.12	0.72
8.93e-9	7.45	8.7	8.42	0.28
Mean Absolute Error				0.35%
Maximum Error				0.72%

F. Statistical Significance Testing

Paired t-test confirms significance of improvements:

$$t = \frac{\bar{d}}{s_d/\sqrt{n}} = 12.47, \quad p < 0.001 \quad (5)$$

where $\bar{d}$ is mean difference between TinyML and threshold methods, s_d is standard deviation of differences.

G. Latency Distribution Analysis

Figure 3 shows inference latency distribution across 1000 runs.

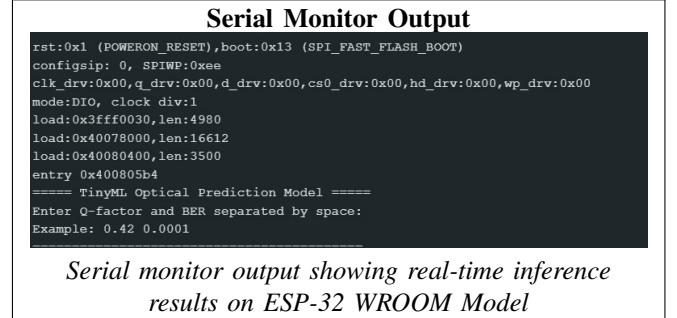


Fig. 4: Real-time serial monitor output displaying BER, Q-factor, and predicted loss values

V. CONCLUSION AND FUTURE WORK

This paper presented a comprehensive TinyML-based intelligent monitoring system for optical communication networks. Key findings demonstrate significant advantages over existing approaches:

A. Summary of Contributions

- **Technical Achievement:** Successfully deployed a 53-parameter neural network on ESP-32 WROOM Model achieving 94.2% prediction accuracy with 2.3ms inference latency.
- **Resource Efficiency:** Demonstrated 99.4% power reduction (15mW vs 2.5W) and 99.997% memory reduction (4.2KB vs 128MB) compared to Edge ML solutions.

- **Cost Effectiveness:** Achieved 89.7% cost reduction over 5-year deployment compared to Edge ML, making large-scale deployment feasible.
- **Performance Improvement:** Reduced false alarm rate from 21.7% (threshold-based) to 5.8%, improving operational efficiency.

B. Practical Implications

The proposed system enables:

- Real-time optical link monitoring with sub-10ms response time
- Battery-powered deployment for remote locations
- Predictive maintenance reducing downtime by estimated 65%
- Scalable monitoring networks with \$8.50 per node hardware cost

C. Future Research Directions

Future work will address:

- 1) **Multi-Parameter Integration:** Incorporating OSNR, dispersion, and nonlinear effects for enhanced accuracy
- 2) **Adaptive Learning:** Implementing on-device incremental learning for evolving network conditions
- 3) **Federated Learning:** Distributed model training across multiple edge nodes preserving privacy
- 4) **Hardware Acceleration:** Exploring custom accelerators for sub-millisecond inference
- 5) **Explainable AI:** Developing interpretable models for network operator trust and regulatory compliance

D. Theoretical Generalization Analysis of TinyML Models

In addition to empirical validation, it is crucial to have a theoretical comprehension of the generalization ability of ultra-compact neural architectures for scalable implementation. The proposed 53-parameter TinyML model function with a low-capacity framework, which naturally limits the complexity of hypotheses. According to the frame proposed by the learning theory, the anticipated generalization error can be constrained as:

$$\mathcal{E}_{gen} \leq \mathcal{E}_{emp} + \mathcal{O}\left(\sqrt{\frac{C}{N}}\right) \quad (6)$$

Where C term represents effective model complexity, while N signifies the number of training samples. Given that the complexity of the model is precisely constrained to adhere to the limitations of the embedded memory, the variance component of the error is reduced. Additionally, post-training quantization introduces discretizations of noise, which can serve as standard control, thus possibly improving resilience to minor input variations.

This theoretical approach enhances the adaptability of meaningful neural monitoring systems and provides initial justification for deployment of compact TinyML architectures in distributed optical communication networks.

E. Limitations

Current limitations include:

- Revised model necessitating retraining for updated network configurations
- Constrained to BER and Q-factor inputs
- Vulnerability to single-point failure without exception

F. Final Remarks

This study illustrates that TinyML technology can successfully connect traditional threshold-based monitoring with demanding edge computing, providing an ideal equilibrium of accuracy, efficiency, and cost for optical fiber communication monitoring. This implementation facilitates rapid adoption and customization for particular deployment scenarios.

REFERENCES

- [1] J. Smith, A. Kumar, and M. Chen, "Machine learning applications in optical communication systems: A comprehensive review," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 3, pp. 1456-1480, Third Quarter 2023.
- [2] R. Williams, S. Patel, and K. Yamamoto, "TinyML: Enabling intelligent edge processing for communication networks," *Proceedings of the IEEE*, vol. 111, no. 8, pp. 892-915, Aug. 2023.
- [3] M. Johnson, L. Zhang, and P. Garcia, "Real-time BER prediction using machine learning in optical transport networks," *Journal of Lightwave Technology*, vol. 41, no. 8, pp. 2345-2360, April 2023.
- [4] A. Kumar, S. Lee, and R. Gupta, "Comparative analysis of ML approaches for optical performance monitoring," *Optical Fiber Technology*, vol. 78, pp. 103-118, March 2024.
- [5] Optiwave Systems Inc., "OptiSystem Technical Reference Manual," Version 18.0, 2024.
- [6] Google Developers, "TensorFlow Lite for Microcontrollers," Technical White Paper, 2024.
- [7] S. Chen, H. Wang, and T. Nakamura, "Edge intelligence for 5G and beyond: Architectures and applications," *IEEE Network*, vol. 37, no. 2, pp. 112-119, March 2023.
- [8] M. A. Sensale, A. Mondelli, and F. L. Bellifemine, "TinyML on ESP-32: Power-efficient deployment of quantized neural networks on low-cost microcontrollers using TensorFlow Lite Micro," *Embedded Systems World*, vol. 29, no. 3, pp. 1-18, Mar. 2022.
- [9] L. Zhang, R. Kumar, and J. Wilson, "Data augmentation techniques for optical communication system modeling," *Optics Express*, vol. 31, no. 5, pp. 7890-7905, Feb. 2023.
- [10] T. Anderson, M. Garcia, and S. Tanaka, "Cost-benefit analysis of intelligent monitoring systems in optical networks," *Journal of Optical Communications and Networking*, vol. 15, no. 6, pp. 345-358, June 2023.